IMDB Data Analysis with Bernoulli Naive Bayes Classifier

Part A

Importing Libraries

In [1]:
```
import numpy as np
import tensorflow as tf
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics import precision_recall_fscore_support, classification_report, accuracy_score
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
```

Loading and Preprocessing IMDB Dataset

In [2]:
```
# Optimal Hyperparameters
m = 3000  # Number of words in the vocabulary
n = 50    # Skip top-N most frequent words
k = 20    # Skip bottom-K least frequent words

g = 800   # Number of most important features to keep (Information Gain)
infogain = False  # Set to True to enable Information Gain feature selection
hyperparameter_tuning = False  # Set to True to execute hyperparameter tuning

# Load IMDB dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.imdb.load_data(num_words=m-k, skip_top=n)
word_index = tf.keras.datasets.imdb.get_word_index()

# Decode indices back to words
def decode_review(encoded_review, index2word):
    return ' '.join([index2word.get(idx, '?') for idx in encoded_review])

index2word = {i + 3: word for word, i in word_index.items()}
index2word.update({0: '[pad]', 1: '[bos]', 2: '[oov]'})

x_train = np.array([decode_review(review, index2word) for review in x_train])
x_test = np.array([decode_review(review, index2word) for review in x_test])

print("Training samples:", len(x_train))
print("Test samples:", len(x_test))
```

```
Training samples: 25000
Test samples: 25000
```

Creating Binary Vectors

In [3]:
```
# Convert text to binary vectors
binary_vectorizer = CountVectorizer(binary=True)
x_train_binary = binary_vectorizer.fit_transform(x_train).toarray()
x_test_binary = binary_vectorizer.transform(x_test).toarray()

print("Vocabulary size:", len(binary_vectorizer.vocabulary_))
```

```
Vocabulary size: 2873
```

Applying Information Gain for Feature Selection

In [4]:
```
if infogain:
    from math import log

    def IG(class_, feature):
        classes = set(class_)
```

```
        Hc = 0
        for c in classes:
            pc = list(class_).count(c) / len(class_)
            Hc += -pc * log(pc, 2)
        feature_values = set(feature)

        Hc_feature = 0
        for feat in feature_values:
            # pf --> P(X=x)
            pf = list(feature).count(feat) / len(feature)
            indices = [i for i in range(len(feature)) if feature[i] == feat]
            clasess_of_feat = [class_[i] for i in indices]
            for c in classes:
                # pcf --> P(C=c|X=x)
                pcf = clasess_of_feat.count(c) / len(clasess_of_feat)
                if pcf != 0:
                    # - P(X=x) * P(C=c|X=x) * log2(P(C=c|X=x))
                    temp_H = -pf * pcf * log(pcf, 2)
                    # sum for all values of C (class) and X (values of specific feature)
                    Hc_feature += temp_H
        ig = Hc - Hc_feature
        return ig

    # Compute IG for each feature
    ig = [(i, IG(y_train, x_train_binary[:, i])) for i in range(len(x_train_binary[0]))]
    ig.sort(key=lambda x: x[1], reverse=True)  # Sort by IG in descending order
    ig = ig[:g]  # Select top g features

    # Reduce dimensions for training and test data
    x_train_binary = x_train_binary[:, [i[0] for i in ig]]
    x_test_binary = x_test_binary[:, [i[0] for i in ig]]

print("Reduced feature set size:", x_train_binary.shape[1])
```

```
Reduced feature set size: 2873
```

Implementing Bernoulli Naive Bayes Classifier

In [5]:

```
class BernoulliNaiveBayes:
    def __init__(self):
        pass

    def fit(self, X, y):
        # Calculate the a-priori (log) probability for each class
        self.p_log_pos = np.log(np.sum(y) / len(y))
        self.p_log_neg = np.log(1 - np.sum(y) / len(y))

        # Calculate the (log) probability of each set of features given each class (y = 1 (Good) or y = 0
        self.p_log_pos_feature1 = np.log((np.sum(X[y == 1], axis=0) + 1) / (np.sum(y) + 2))
        self.p_log_neg_feature1 = np.log((np.sum(X[y == 0], axis=0) + 1) / (len(y) - np.sum(y) + 2))
        self.p_log_pos_feature0 = np.log(1.0 - (np.sum(X[y == 1], axis=0) + 1) / (np.sum(y) + 2))
        self.p_log_neg_feature0 = np.log(1.0 - (np.sum(X[y == 0], axis=0) + 1) / (len(y) - np.sum(y) + 2)

    def predict(self, X):
        X_reverse = np.where((X == 0) | (X == 1), X ^ 1, X)
        p_log_pos = X_reverse.dot(self.p_log_pos_feature0) + X.dot(self.p_log_pos_feature1) + self.p_log_p
        p_log_neg = X_reverse.dot(self.p_log_neg_feature0) + X.dot(self.p_log_neg_feature1) + self.p_log_n

        return np.array([1 if p_log_pos[i] > p_log_neg[i] else 0 for i in range(len(X))])
```

Hyperparameter Tuning (Optional)

In [6]:

```
if hyperparameter_tuning:
    N = [50, 100, 200, 300]
    K = [0, 20, 50, 80]
    M = [500, 2000, 2500, 3000]

    best_params = {}
    max_accuracy = 0

    for m in M:
        for n in N:
            for k in K:
```

```
                (x_train_temp, y_train_temp), (_, _) = tf.keras.datasets.imdb.load_data(num_words=m-k, sk
                x_train_temp, x_val_temp, y_train_temp, y_val_temp = train_test_split(x_train_temp, y_tra

                # Decode and vectorize
                index2word_temp = {i + 3: word for word, i in word_index.items()}
                index2word_temp.update({0: '[pad]', 1: '[bos]', 2: '[oov]'})

                x_train_temp = np.array([' '.join([index2word_temp.get(idx, '?') for idx in text]) for te
                x_val_temp = np.array([' '.join([index2word_temp.get(idx, '?') for idx in text]) for text

                x_train_binary_temp = binary_vectorizer.fit_transform(x_train_temp).toarray()
                x_val_binary_temp = binary_vectorizer.transform(x_val_temp).toarray()

                model = BernoulliNaiveBayes()
                model.fit(x_train_binary_temp, y_train_temp)
                val_accuracy = accuracy_score(y_val_temp, model.predict(x_val_binary_temp))

                if val_accuracy > max_accuracy:
                    max_accuracy = val_accuracy
                    best_params = {"m": m, "n": n, "k": k}

    print("Best Parameters:", best_params)
    print("Best Validation Accuracy:", max_accuracy)
```

Training and Evaluating the Model

In [7]:
```
# Train the model
bnb = BernoulliNaiveBayes()
bnb.fit(x_train_binary, y_train)

# Evaluate on test data
y_pred_test = bnb.predict(x_test_binary)

# Display results
print("Test set evaluation:")
print(classification_report(y_test, y_pred_test))

# Compute metrics
precision, recall, f1, _ = precision_recall_fscore_support(y_test, y_pred_test, average=None)
micro_avg = precision_recall_fscore_support(y_test, y_pred_test, average="micro")
macro_avg = precision_recall_fscore_support(y_test, y_pred_test, average="macro")

# Display metrics
print("Precision (class-wise):", precision)
print("Recall (class-wise):", recall)
print("F1-score (class-wise):", f1)
print("Micro-averaged metrics (Precision, Recall, F1):", micro_avg[:3])
print("Macro-averaged metrics (Precision, Recall, F1):", macro_avg[:3])
```

```
Test set evaluation:
              precision    recall  f1-score   support

           0       0.85      0.85      0.85     12500
           1       0.85      0.85      0.85     12500

    accuracy                           0.85     25000
   macro avg       0.85      0.85      0.85     25000
weighted avg       0.85      0.85      0.85     25000

Precision (class-wise): [0.84606177 0.8477825 ]
Recall (class-wise): [0.84816 0.84568]
F1-score (class-wise): [0.84710958 0.84672995]
Micro-averaged metrics (Precision, Recall, F1): (0.84692, 0.84692, 0.84692)
Macro-averaged metrics (Precision, Recall, F1): (0.8469221337098911, 0.84692, 0.84691976462383)
```

Learning Curves: Precision, Recall, and F1

In [8]:
```
def plot_learning_curves(model, x_train, y_train, x_dev, y_dev, class_label):
    train_sizes = np.linspace(0.1, 1.0, 10)
    precision_vals, recall_vals, f1_vals = [], [], []
    dev_precision_vals, dev_recall_vals, dev_f1_vals = [], [], []

    for frac in train_sizes:
```

```
        subset_size = int(frac * len(y_train))
        x_subset, y_subset = x_train[:subset_size], y_train[:subset_size]

        model.fit(x_subset, y_subset)

        # Predictions on training subset
        y_pred_train = model.predict(x_subset)
        precision, recall, f1, _ = precision_recall_fscore_support(y_subset, y_pred_train, average=None)
        precision_vals.append(precision[class_label])
        recall_vals.append(recall[class_label])
        f1_vals.append(f1[class_label])

        # Predictions on development set
        y_pred_dev = model.predict(x_dev)
        dev_precision, dev_recall, dev_f1, _ = precision_recall_fscore_support(y_dev, y_pred_dev, average=
        dev_precision_vals.append(dev_precision[class_label])
        dev_recall_vals.append(dev_recall[class_label])
        dev_f1_vals.append(dev_f1[class_label])

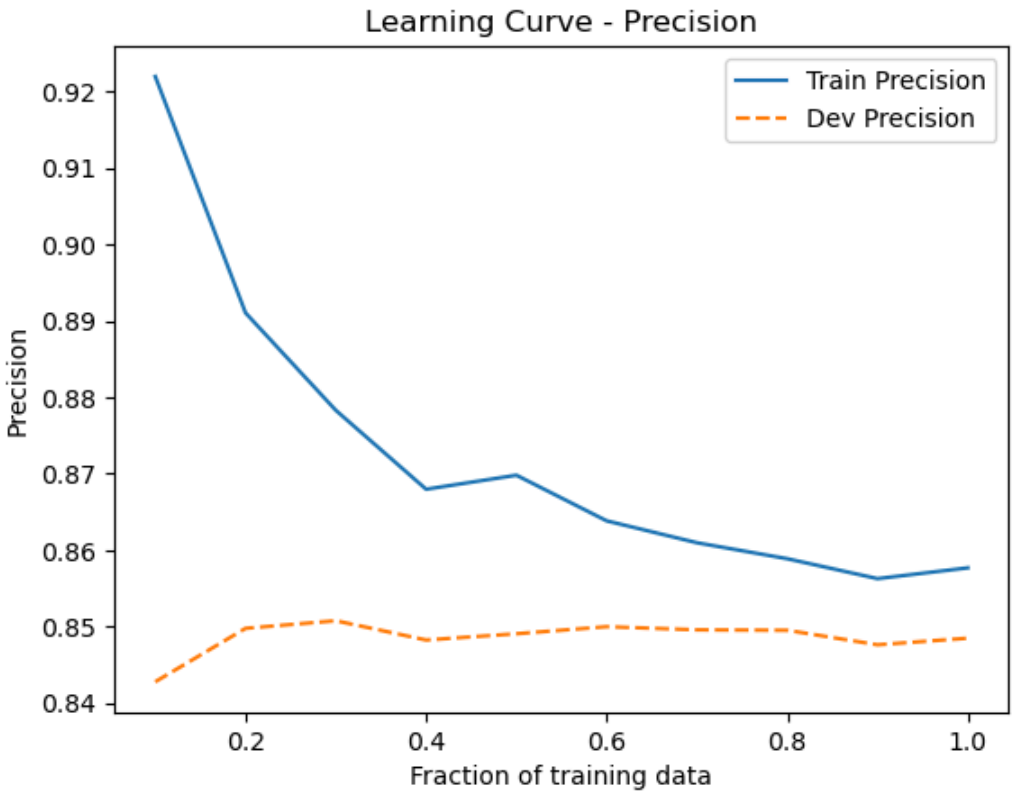
    # Plotting Precision
    plt.figure()
    plt.plot(train_sizes, precision_vals, label="Train Precision")
    plt.plot(train_sizes, dev_precision_vals, label="Dev Precision", linestyle="--")
    plt.xlabel("Fraction of training data")
    plt.ylabel("Precision")
    plt.title("Learning Curve - Precision")
    plt.legend()
    plt.show()

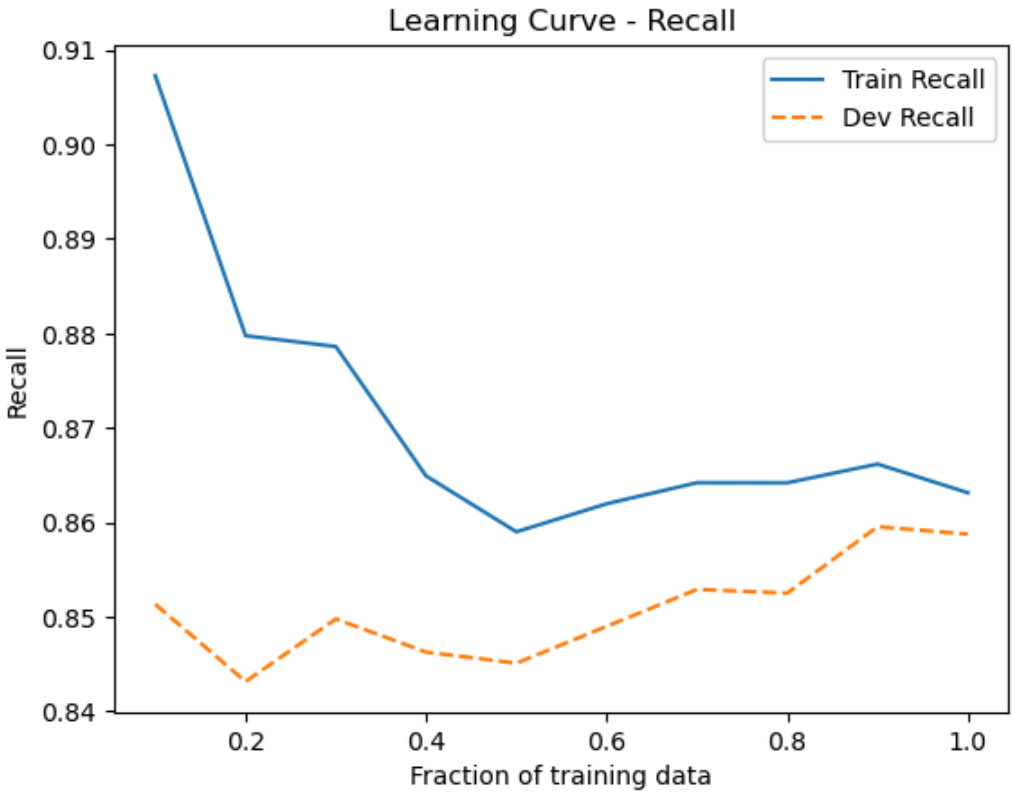
    # Plotting Recall
    plt.figure()
    plt.plot(train_sizes, recall_vals, label="Train Recall")
    plt.plot(train_sizes, dev_recall_vals, label="Dev Recall", linestyle="--")
    plt.xlabel("Fraction of training data")
    plt.ylabel("Recall")
    plt.title("Learning Curve - Recall")
    plt.legend()
    plt.show()

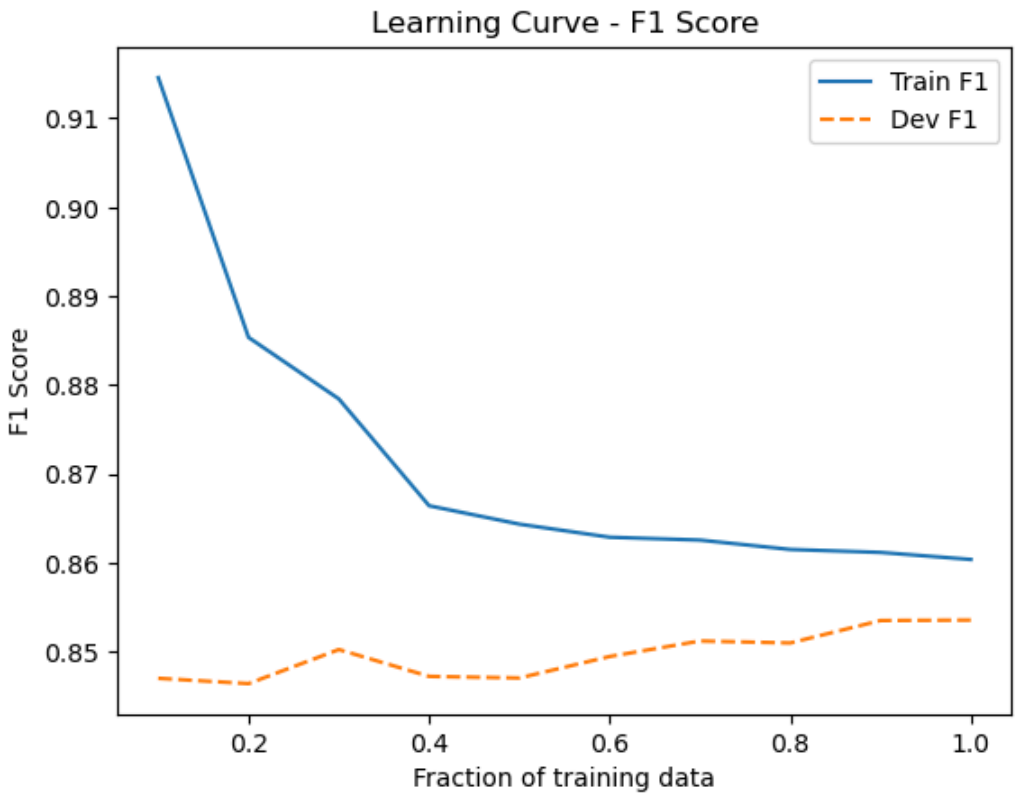
    # Plotting F1 Score
    plt.figure()
    plt.plot(train_sizes, f1_vals, label="Train F1")
    plt.plot(train_sizes, dev_f1_vals, label="Dev F1", linestyle="--")
    plt.xlabel("Fraction of training data")
    plt.ylabel("F1 Score")
    plt.title("Learning Curve - F1 Score")
    plt.legend()
    plt.show()

    # Split training data for learning curve evaluation
x_train_subset, x_dev, y_train_subset, y_dev = train_test_split(x_train_binary, y_train, test_size=0.2, ra

# Plot learning curves for class 1 (positive reviews)
plot_learning_curves(bnb, x_train_subset, y_train_subset, x_dev, y_dev, class_label=1)
```


Learning Curve - Precision
Train Precision
Dev Precision
Precision
0.92
0.91
0.90
0.89
0.88
0.87
0.86
0.85
0.84
0.2
0.4
0.6
0.8
1.0
Fraction of training data


Learning Curve - Recall
Train Recall
Dev Recall
Recall
0.91
0.90
0.89
0.88
0.87
0.86
0.85
0.84
0.2
0.4
0.6
0.8
1.0
Fraction of training data



Part B

Comparing with Scikit-learn Implementations

In [9]:
```
# Importing Libraries
from sklearn.naive_bayes import BernoulliNB
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
```

Bernoulli Naive Bayes against Scikit-learn Bernoulli Naive Bayes

In [10]:
```
# Scikit-learn Bernoulli Naive Bayes
bnb_sklearn = BernoulliNB()
bnb_sklearn.fit(x_train_binary, y_train)

# Evaluate on test data
y_pred_test_sklearn = bnb_sklearn.predict(x_test_binary)

# Display results
print("Scikit-learn Bernoulli Naive Bayes - Test set evaluation:")
print(classification_report(y_test, y_pred_test_sklearn))
```

```
Scikit-learn Bernoulli Naive Bayes - Test set evaluation:
              precision    recall  f1-score   support

           0       0.85      0.85      0.85     12500
           1       0.85      0.85      0.85     12500

    accuracy                           0.85     25000
   macro avg       0.85      0.85      0.85     25000
weighted avg       0.85      0.85      0.85     25000
```

Bernoulli Naive Bayes against Scikit-learn Random Forest Classifier

In [11]:
```
# Random Forest Classifier
#rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf = RandomForestClassifier(n_estimators=100, max_depth=10, min_samples_split=5, random_state=42)

rf.fit(x_train_binary, y_train)
```

```
# Evaluate on test data
y_pred_test_rf = rf.predict(x_test_binary)

# Display results
print("Random Forest Classifier - Test set evaluation:")
print(classification_report(y_test, y_pred_test_rf))
```

```
Random Forest Classifier - Test set evaluation:
              precision    recall  f1-score   support

           0       0.85      0.77      0.81     12500
           1       0.79      0.87      0.83     12500

    accuracy                           0.82     25000
   macro avg       0.82      0.82      0.82     25000
weighted avg       0.82      0.82      0.82     25000
```

Bernoulli Naive Bayes against Scikit-learn Logistic Regression

In [12]:
```
# Logistic Regression
log_reg = LogisticRegression(max_iter=1000, random_state=42)
log_reg.fit(x_train_binary, y_train)

# Evaluate on test data
y_pred_test_log_reg = log_reg.predict(x_test_binary)

# Display results
print("Logistic Regression - Test set evaluation:")
print(classification_report(y_test, y_pred_test_log_reg))
```

```
Logistic Regression - Test set evaluation:
              precision    recall  f1-score   support

           0       0.86      0.86      0.86     12500
           1       0.86      0.86      0.86     12500

    accuracy                           0.86     25000
   macro avg       0.86      0.86      0.86     25000
weighted avg       0.86      0.86      0.86     25000
```

Summary of Results

In [13]:
```
# Summary Table of Results
models = ["Bernoulli NB", "Scikit-learn Bernoulli NB", "Scikit-learn Random Forest", "Scikit-learn Logist
results = [
    classification_report(y_test, y_pred_test, output_dict=True),
    classification_report(y_test, y_pred_test_sklearn, output_dict=True),
    classification_report(y_test, y_pred_test_rf, output_dict=True),
    classification_report(y_test, y_pred_test_log_reg, output_dict=True)
]

for model, result in zip(models, results):
    print("\nModel:", model)
    print("Accuracy:", result['accuracy'])
    print("Precision (weighted avg):", result['weighted avg']['precision'])
    print("Recall (weighted avg):", result['weighted avg']['recall'])
    print("F1-Score (weighted avg):", result['weighted avg']['f1-score'])
```

```
Model: Bernoulli NB
Accuracy: 0.84692
Precision (weighted avg): 0.8469221337098912
Recall (weighted avg): 0.84692
F1-Score (weighted avg): 0.84691976462383

Model: Scikit-learn Bernoulli NB
Accuracy: 0.84692
Precision (weighted avg): 0.8469221337098912
Recall (weighted avg): 0.84692
F1-Score (weighted avg): 0.84691976462383

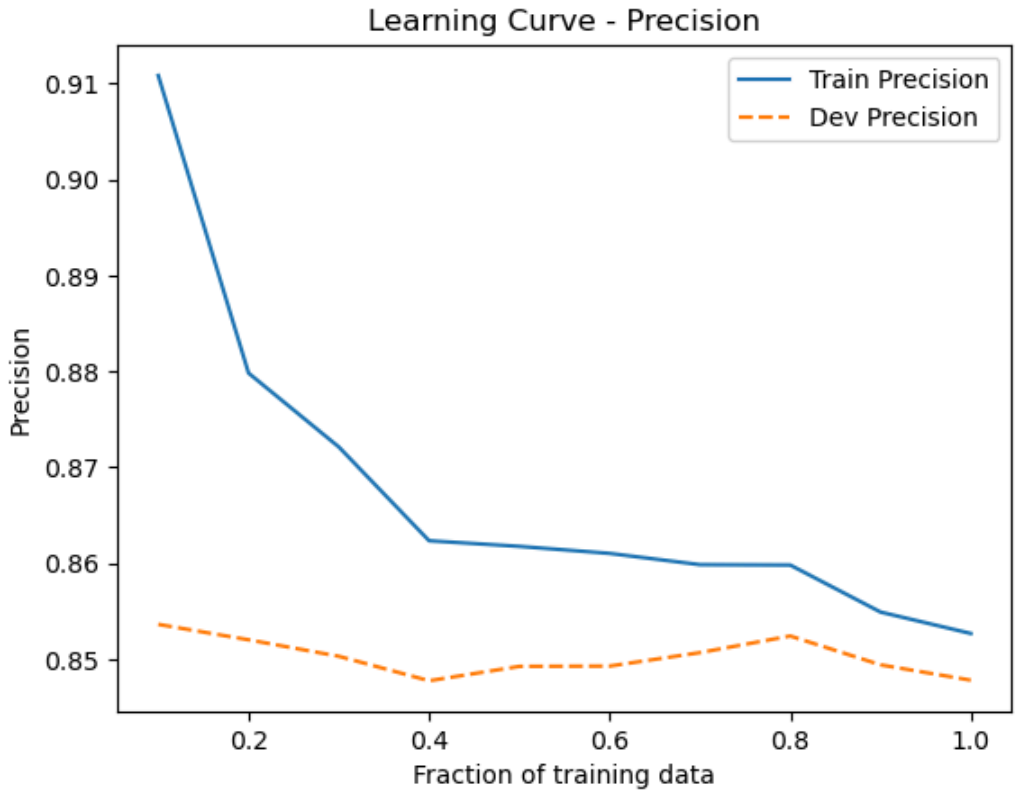
Model: Scikit-learn Random Forest
Accuracy: 0.81912
Precision (weighted avg): 0.822138515200674
Recall (weighted avg): 0.81912
F1-Score (weighted avg): 0.8186952828217269

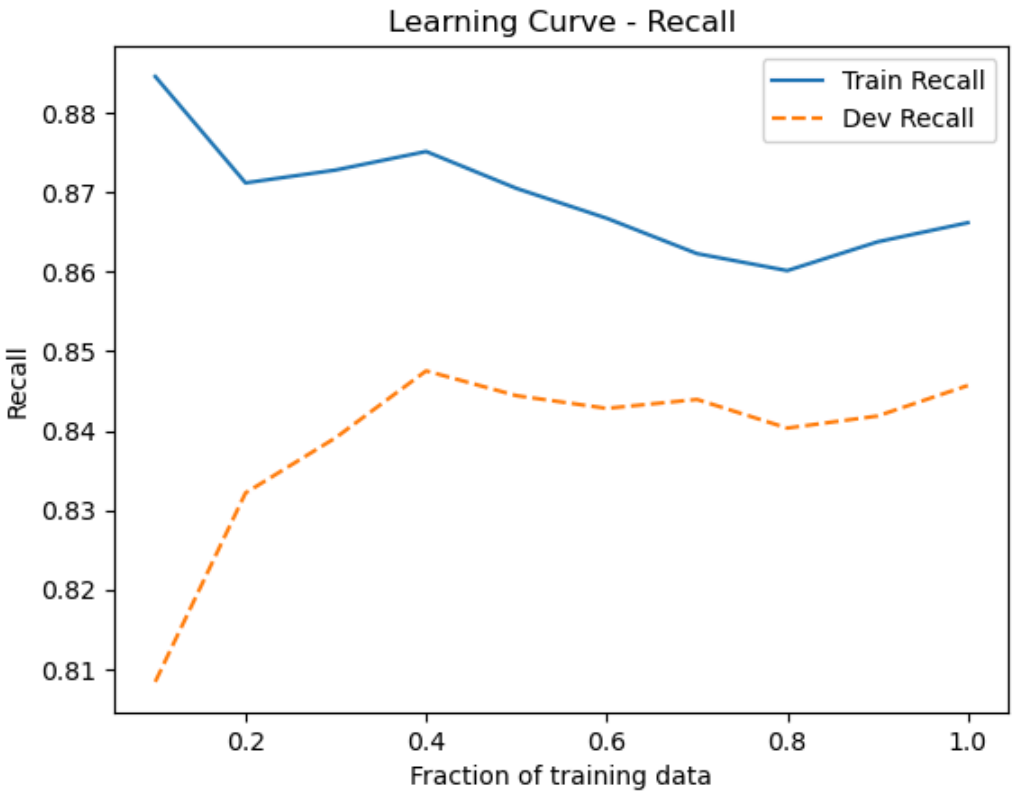
Model: Scikit-learn Logistic Regression
Accuracy: 0.86
Precision (weighted avg): 0.8600119443322843
Recall (weighted avg): 0.86
F1-Score (weighted avg): 0.8599988387743683
```

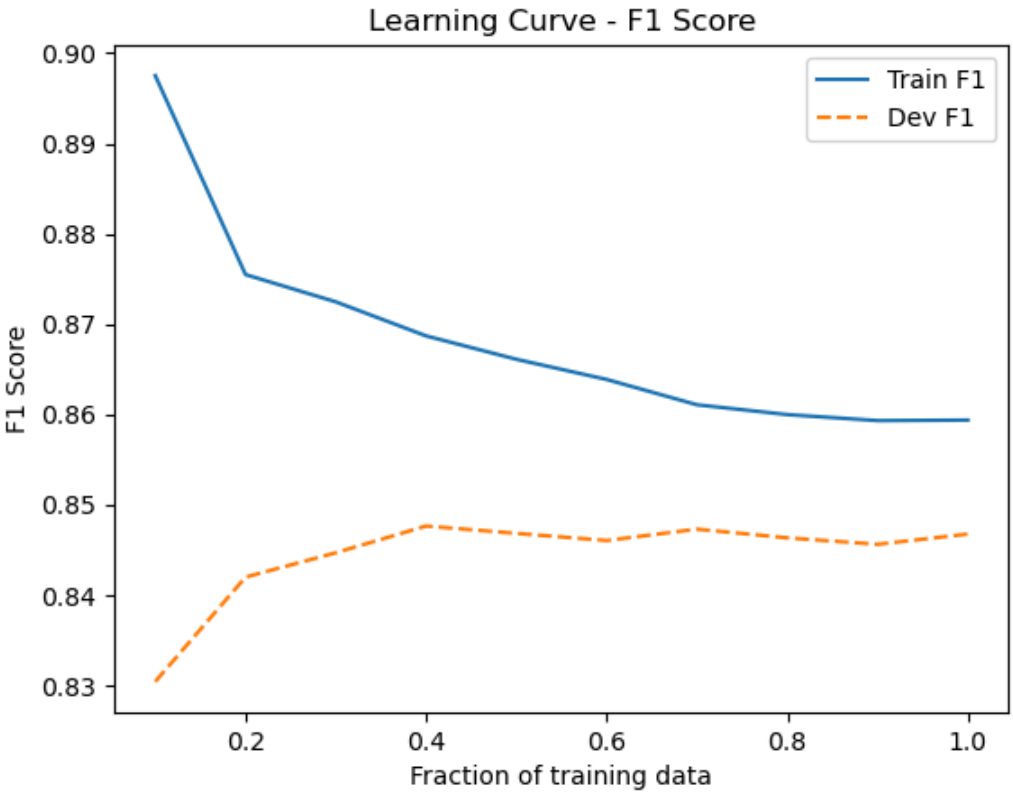
Learning Curves: Precision, Recall, and F1

Scikit-learn Bernoulli Naive Bayes

In [14]:
```
plot_learning_curves(bnb_sklearn, x_train_binary, y_train, x_test_binary, y_test, class_label=1)
```

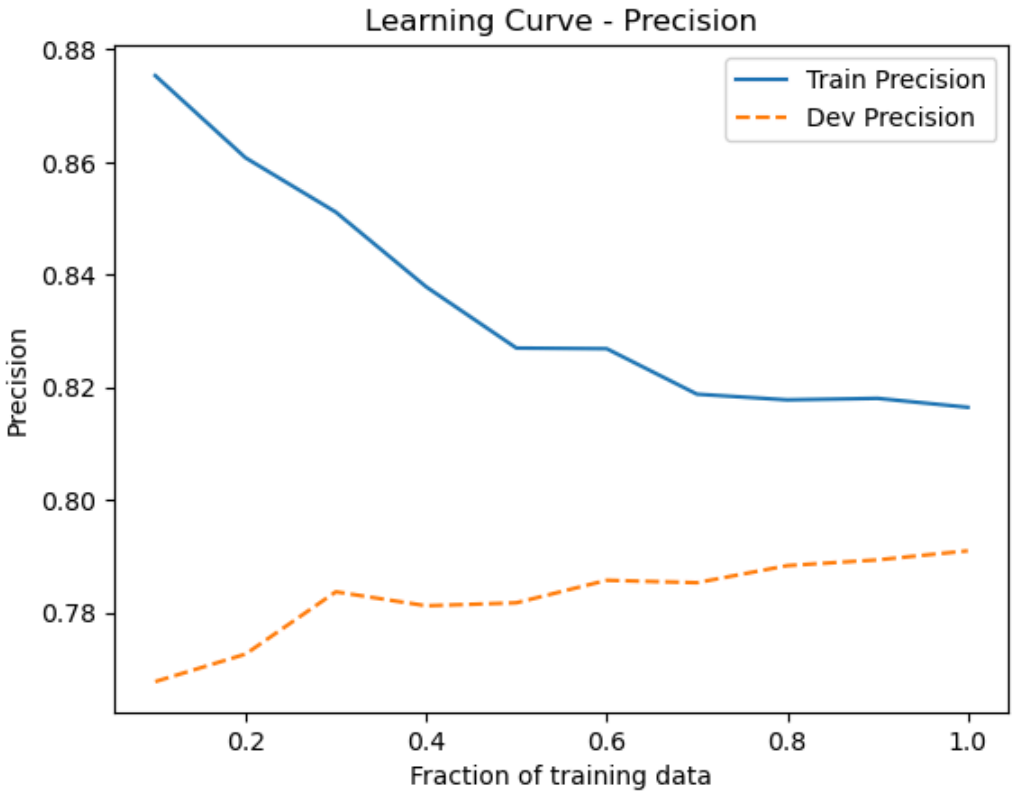


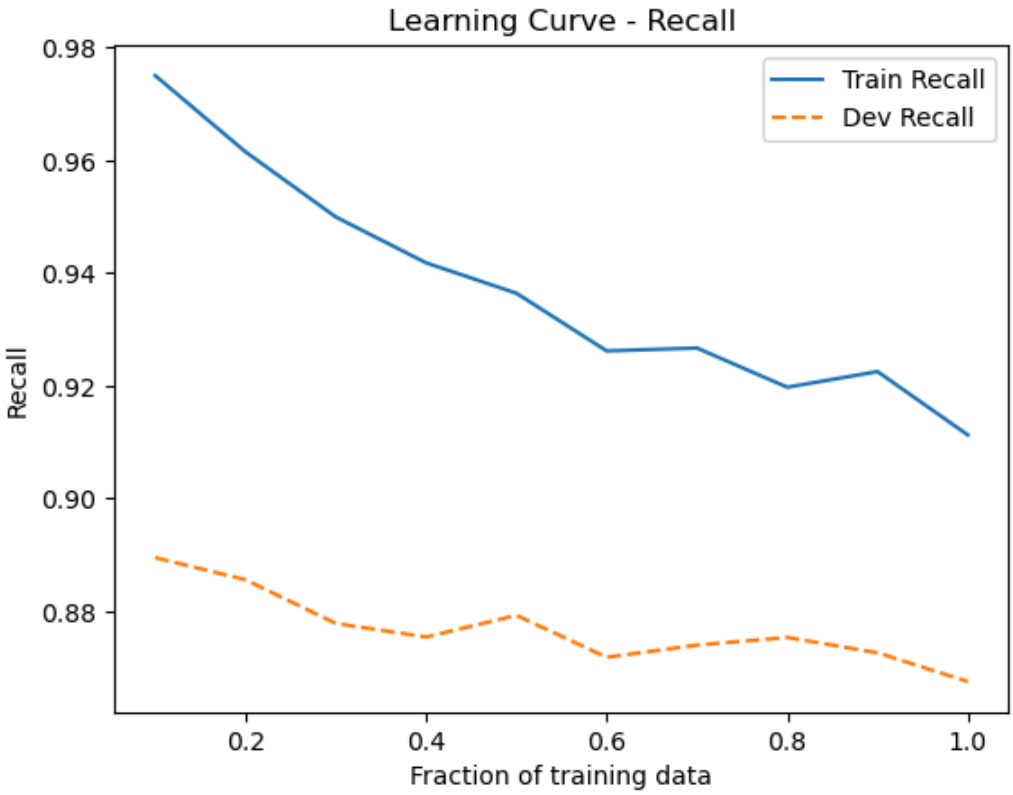


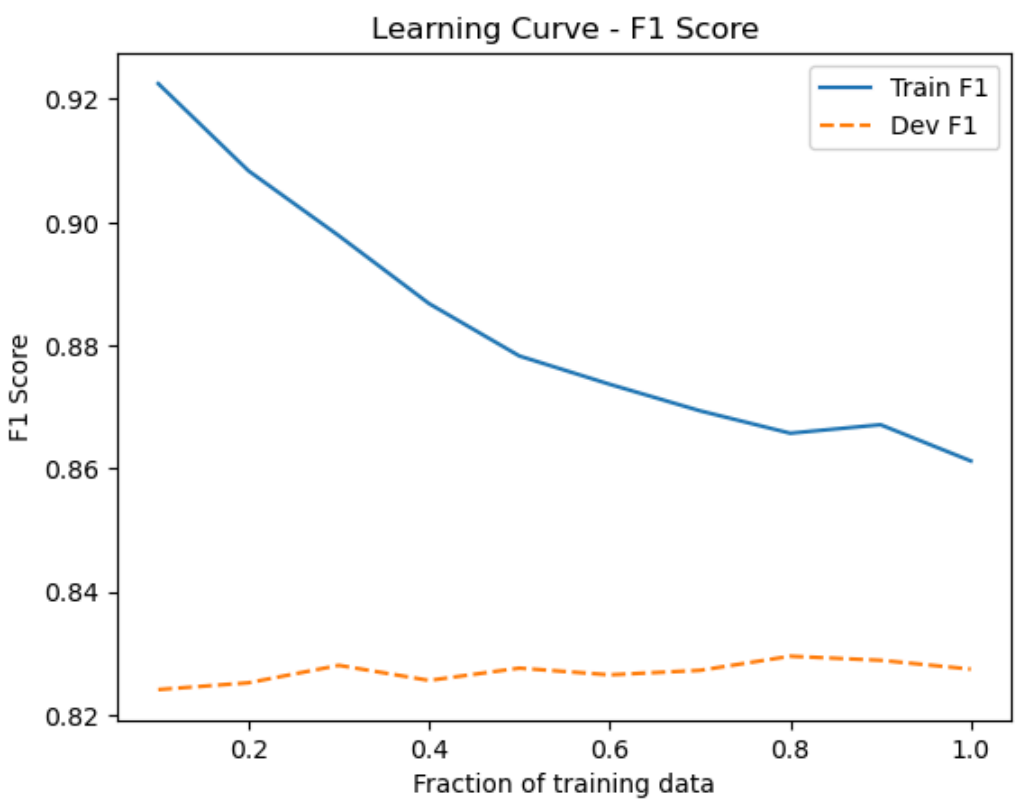


Scikit-learn Random Forest

In [15]:
```
plot_learning_curves(rf, x_train_binary, y_train, x_test_binary, y_test, class_label=1)
```

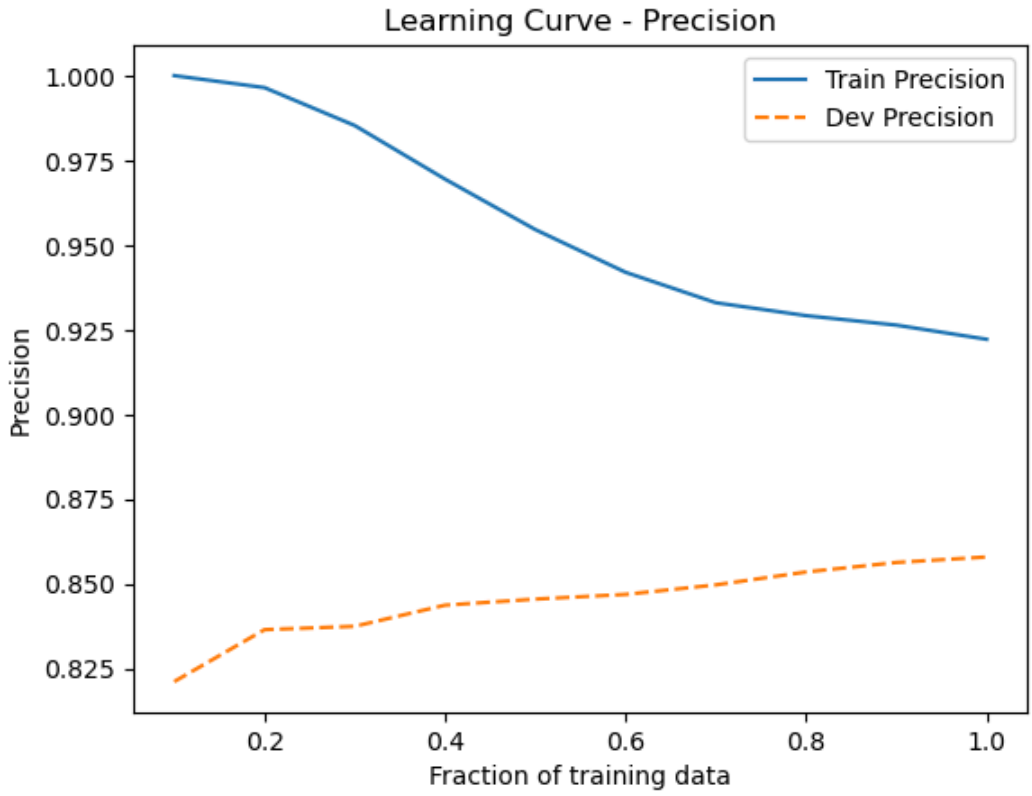

Learning Curve - Precision
Train Precision
Dev Precision
Precision
0.88
0.86
0.84
0.82
0.80
0.78
0.2
0.4
0.6
0.8
1.0
Fraction of training data

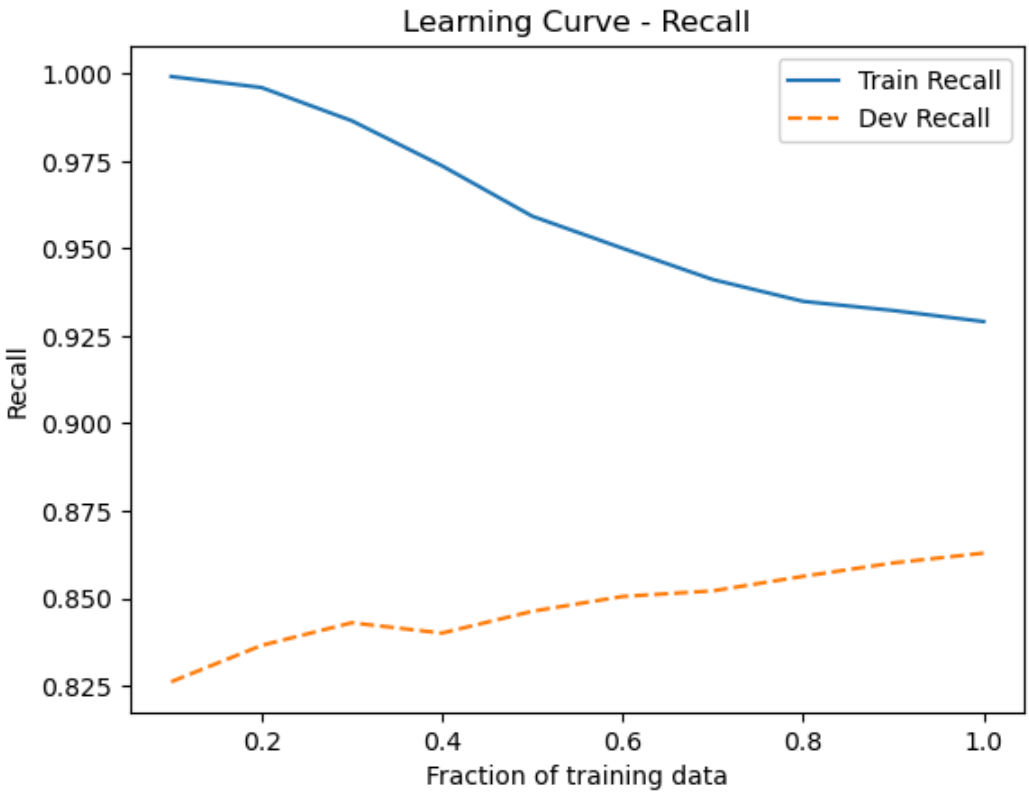

Learning Curve - Recall
Train Recall
Dev Recall
Recall
0.98
0.96
0.94
0.92
0.90
0.88
0.2
0.4
0.6
0.8
1.0
Fraction of training data

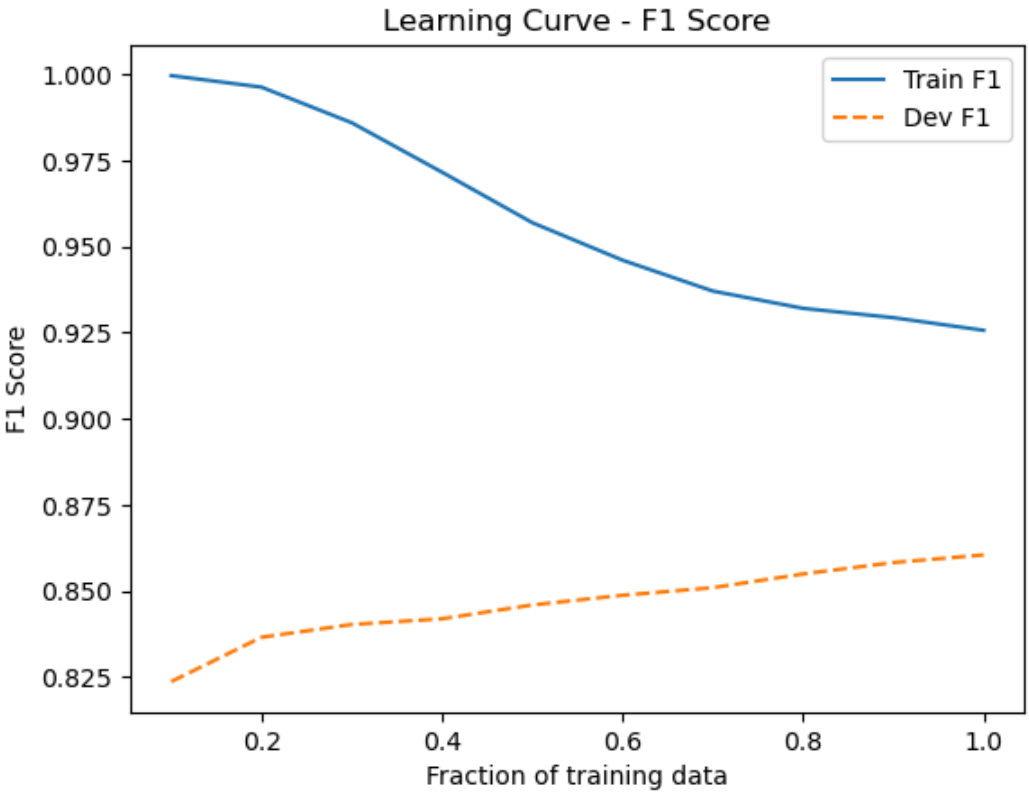


Scikit-learn Logistic Regression

In [16]:
```
plot_learning_curves(log_reg, x_train_binary, y_train, x_test_binary, y_test, class_label=1)
```




Learning Curve - Recall
Train Recall
Dev Recall
Recall
1.000
0.975
0.950
0.925
0.900
0.875
0.850
0.825
0.2
0.4
0.6
0.8
1.0
Fraction of training data


Learning Curve - F1 Score
Train F1
Dev F1
F1 Score
1.000
0.975
0.950
0.925
0.900
0.875
0.850
0.825
0.2
0.4
0.6
0.8
1.0
Fraction of training data